

DS n° 2**CORRECTION****EXERCICE 1 : AMÉLIORATION DE COMPLEXITÉ****A - Calcul de somme**

```
1)
1 def factorielle(k) :
2     P = 1
3     for k in range(k):
4         P = P*(k+1)
5     return P
```

2) On fait une multiplication par passage en boucle, et k passages en boucle, donc k multiplications.

On propose le code suivant :

```
1 def calcul_somme(n):
2     S = 0
3     for k in range(n) :
4         S += factorielle(k)
5     return S
```

3) Au k -ième passage en boucle on fait k multiplications, et k prend les valeurs $0, 1, \dots, n-1$, on fait donc $1 + \dots + n-1 = O(n^2)$ multiplications.

4) On recalcule les factorielle en entier à chaque fois, il serait plus efficace d'utiliser $(k+1) = (k+1)k!$.

```
1 def calcul_mieux(n):
2     S = 0
3     P = 1
4     for k in range(n) :
5         P *= (k+1)
6         S += P
7     return S
```

Il y a ici 1 seule multiplication par passage en boucle, pour un cout linéaire.

B - Puissance rapide

```
5)
1 def puissance1(x,n):
2     p = 1
3     for i in range(n):
4         p = p*x
5     return p
```

6) On utilise que si n est pair $x^n = (x^{n/2})^2$ et si n est impair $x^n = (x^{n/2})^2 \times x$

```
7)
1 def puissance2(x,n):
2     p = 1
3     while n != 0:
4         if n%2 == 1:
5             p = p*x
6             x = x*x
7             n = n//2
8     return p
```

C - Recherche de la plus longue sous-chaîne sans caractère répété

Pour une chaîne de caractère donnée, on veut trouver la longueur de la plus longue sous-chaîne d'une chaîne donnée qui ne contient pas de caractères répétés.

Par exemple pour la chaîne " robot" la longueur maximale est de 3, pour "rob".

Voici une proposition

```

1 def naive_longest_unique_substring(s):
2     max_length = 0
3     n = len(s)
4     for i in range(n):
5         seen = []
6         j = i
7         while j < n :
8             if not s[j] in seen:
9                 seen.append(s[j])
10                j += 1
11            else :
12                j = n
13        max_length = max(max_length, len(seen))
14    return max_length

```

8) A faire quoi.

9) Dans le pire des cas tous les caractères sont différents, et la taille de seen augmente de 1 à chaque passage en ligne 9. Le cout de la ligne 8 est donc de $1, 2, \dots, n - j$, pour j entre i et $n - 1$, pour i entre 0 et $n - 1$, ce qui donne une complexité cubique.

```

10
1 def optimized_longest_unique_substring(s):
2     n = len(s)
3     start = 0
4     max_length = 0
5     seen = [-1] * 256 # Tableau pour indexer les caractères ASCII (128 suffirait pour les caractères
6                        # standards)
7     for end in range(n):
8         char_index = ord(s[end])
9         # Si le caractère est déjà vu dans la fenêtre actuelle
10        if seen[char_index] >= start:
11            start = seen[char_index] + 1
12
13        # Met à jour la position du caractère
14        seen[char_index] = end
15        # Met à jour la longueur maximale
16        max_length = max(max_length, end - start + 1)
17
18    return max_length

```

EXERCICE 2 : PERMUTATIONS DE LISTES

A - Exemples

- 1) L[1]
- 2) [1, 3, 2, 0, 4].

B - Support

```

2)
1 def support(p):
2     return [i for i in range(len(p)) if p[i] != i]

```

- 4) Par exemple $p(0) = 0, p(1) = 3, p(2) = 2, p(3) = 4, p(4) = 1$.

C -Composition des permutations

```
5)
1 def compose(p, q):
2     return [q[p[i]] for i in range(len(p))]
```

- Exemple :

```
1 >>> compose([2, 0, 1], [1, 2, 0])
2 [0, 1, 2]
```

6) `compose(compose(p, q), r) == compose(p, compose(q, r))`

D - Inverse

```
7)
1 def inverse(p):
2     n = len(p)
3     q = [0] * n
4     for i in range(n):
5         q[p[i]] = i
6     return q
```

8) `compose(p, inverse(p)) == [k for k in range(len(p))]`

E - Génération de permutations

9) `[0,1,2]` , `[0,2,1]` , `[1,2,0]` , `[1,0,2]` , `[2,1,0]` , `[2,0,1]`

```
10)
1 def generate_permutations(lst):
2     if len(lst) == 0:
3         return [[]]
4     permutations = []
5     for i in range(len(lst)):
6         remaining = lst[:i] + lst[i+1:]
7         for perm in generate_permutations(remaining):
8             permutations.append([lst[i]] + perm)
9     return permutations
```